



LECTURE 11

NEUROSYMBOLIC LOWERING METHODS:

INFERENCE-TIME, DEEP INTEGRATION OF KNOWLEDGE
GRAPHS WITH LLMS

DATASCI290: NEUROSYMBOLIC AI

FEBRUARY 11 2026

PIVOT

- Lowering approaches
 - Knowledge Graph lowering (into neural networks)
 - Especially at *inference time*
 - Datalog Lowering: “Tensor Logic”
- Frameworks and Tools
 - PyKeen
 - “Programming LLMs”: representation pipeline
 - <https://github.com/SonglinZhai/KGA>
 - Graph Embeddings
 - Node2vec, ClinVec
- Reasoning Models and Limitations
- True Neurosymbolic Models
- And of course ... building CareTrace with a lifting approach that respects hard constraints, does not miss red flags, and is traceable in medical knowledge and guidelines !

LET US UNDERSTAND

- Zhai, S., Qi, G., Wang, Y., & Meng, Y. (2025). *Knowledge Fusion via Bidirectional Information Aggregation*. arXiv preprint arXiv:2507.08704.

MOTIVATION: DYNAMIC KNOWLEDGE VS STATIC LLMS

- LLMs are “static” after pre-training: their internal facts go stale, especially for time-sensitive or evolving knowledge.
- Knowledge graphs (KGs), on the other hand, are “dynamic”: they can be updated continuously with new entities, relations, corrections.
- Goal: Make a (frozen) LLM behave as if it can consult a continuously updated knowledge source **at inference time**.
- Most existing KG+LLM methods either
 - require changing model parameters (risking degradation), or
 - inject KG facts in ways that don't respect KG structure and create noise.



WHY NOT (JUST) FINE-TUNE
(CHANGE MODEL PARAMETERS) ?



WHY NOT FINE-TUNING / PEFT FOR KG KNOWLEDGE?

- PEFT: Parameter-efficient Fine Tuning
- Many approaches “attach” KG knowledge by training (fine-tuning / PEFT).
- The motivation for **avoiding parameter changes**:
 - **Catastrophic forgetting risk**: new adaptation can overwrite older capabilities.
 - **General capability degradation**: the model becomes worse outside the adapted distribution.
 - **Operational mismatch** with dynamic knowledge: KGs update frequently; re-training repeatedly is expensive and brittle.
- Desired properties:
 - **Non-invasive**: keep original attention weights and projections.
 - **Dynamic**: work with a changing set of triples *without training*.



WHY MIGHT “KG IN PROMPT” BE
PROBLEMATIC (OR LIMITED) ?



WHY “KG IN PROMPT” (ICL) IS OFTEN NOT ENOUGH

- Common baseline: flatten triples into text and prepend them to the question (“in-context learning with KG facts”).
- Why this can fail structurally:
 - KG is a **typed graph**; prompt is a **sequence** (structure gets blurred).
 - triples interact with each other in uncontrolled ways (“inter-triple interference”).
 - results become sensitive to triple ordering and prompt length.
- What the paper aims for instead:
 - keep triples as a **separate memory-like object**
 - integrate them via **attention**, not via raw prompt concatenation.

KGA: KNOWLEDGE GRAPH-*GUIDED* ATTENTION

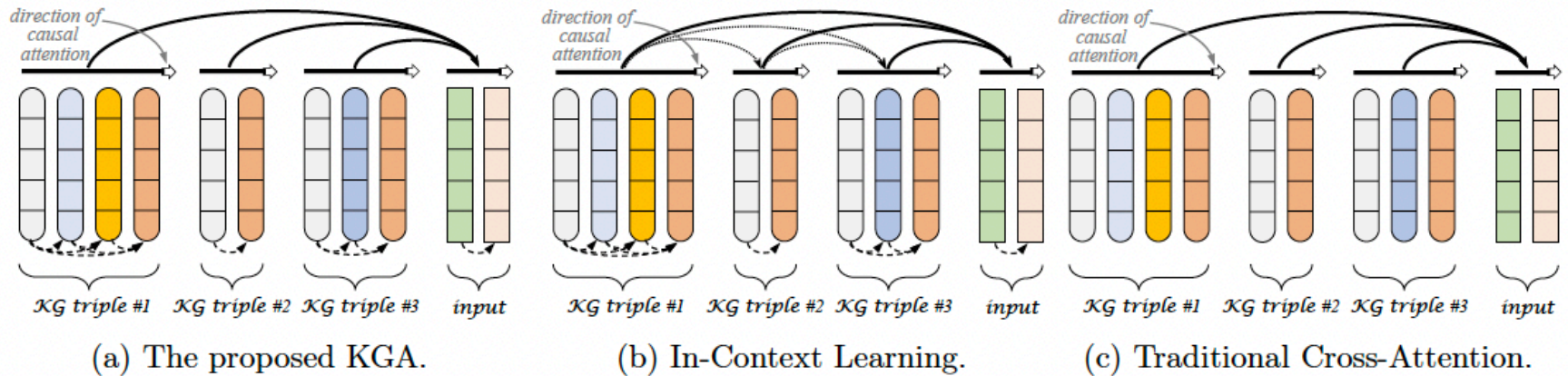


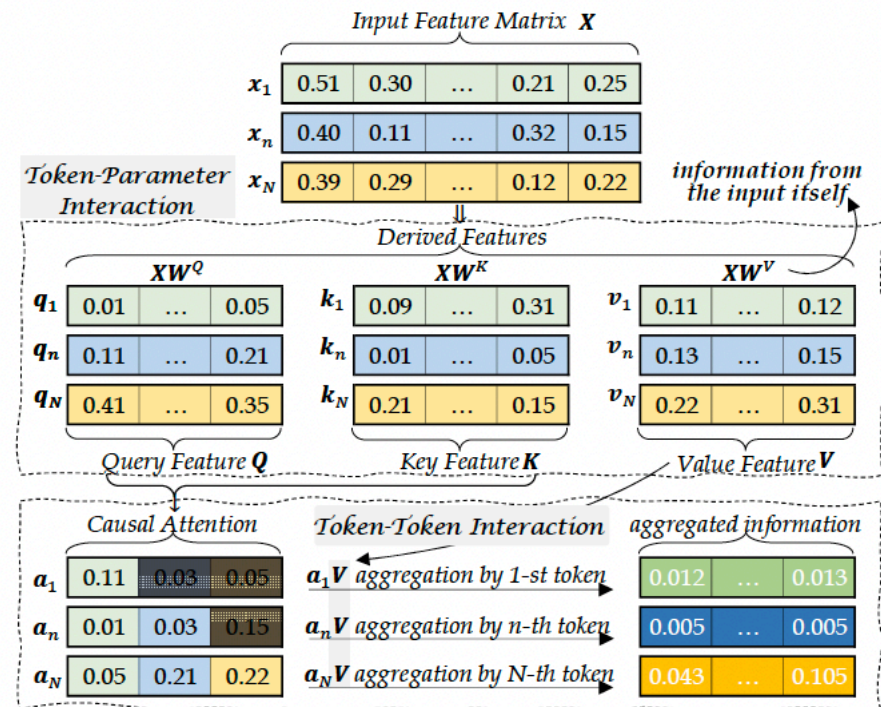
Figure 10: Illustrative depiction of information aggregation under different methods.

BOTTOM-UP AND TOP-DOWN

- Cognitive Science inspiration:
 - Maurizio Corbetta and Gordon L. Shulman. *Control of goal-directed and stimulus-driven attention in the brain*. Nature Reviews Neuroscience, 3:201–215, 2002.
 - **Bottom-up**: stimulus-driven retrieval: the question triggers relevant background knowledge.
 - **Top-down**: goal-directed verification: candidate knowledge is checked against what's needed.
- The core design idea:
 - Let **tokens pull knowledge** (bottom-up fusion),
 - While **knowledge “checks” tokens** to decide relevance (top-down guidance).
- Why bidirectionality matters:
 - bottom-up alone can over-inject noisy or irrelevant triples
 - top-down provides a “relevance filter” that adapts per query

ATTENTION (IN TRANSFORMERS) : RECAP

- At each layer, every token creates:
 - a **Query**: “what am I looking for?”
 - a **Key**: “what do I represent / offer?”
 - a **Value**: “what content do I contribute if attended to?”
- Attention does:
 - compare Query(token i) against Key(token j) for all j
 - turn comparisons into weights
 - mix the Value vectors using those weights
- Layer-by-layer effect: token representations become more contextual and relational as layers accumulate.



SELF ATTENTION

*“The trophy didn’t fit in the suitcase because **it** was too big.”*

*“The **trophy** didn’t fit in the suitcase because it was too big.”*

- The token “it” needs context to resolve what “it” refers to.
- In one attention head:
 - “it” attends strongly to “trophy” (semantics match “too big”).
 - “because” attends to cause-related region.
- Next layer: “it” representation now carries “trophy-like” meaning, enabling correct continuation.
- Attention is a learned routing mechanism that moves information across tokens.

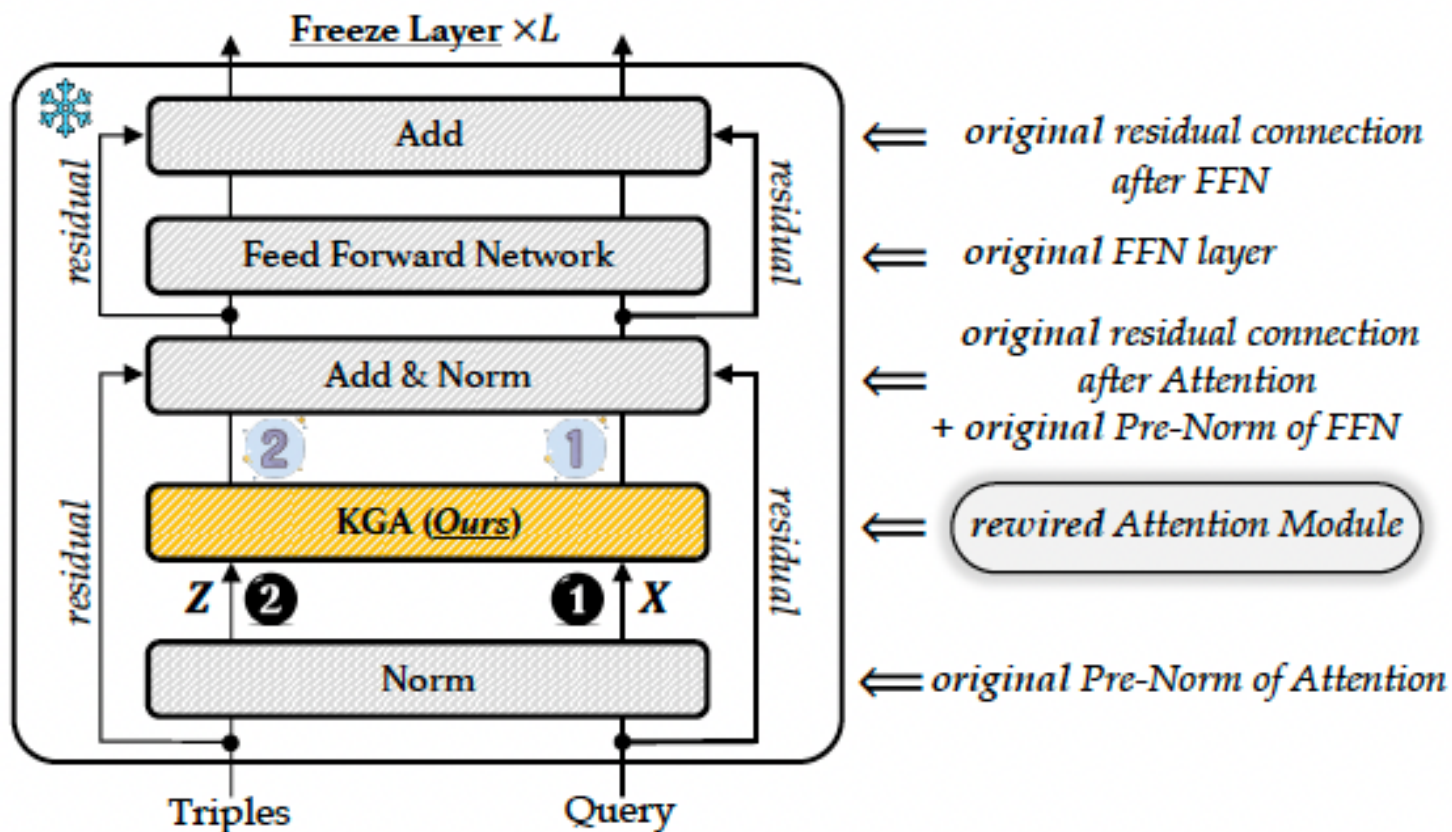
KGA: TASK SETUP

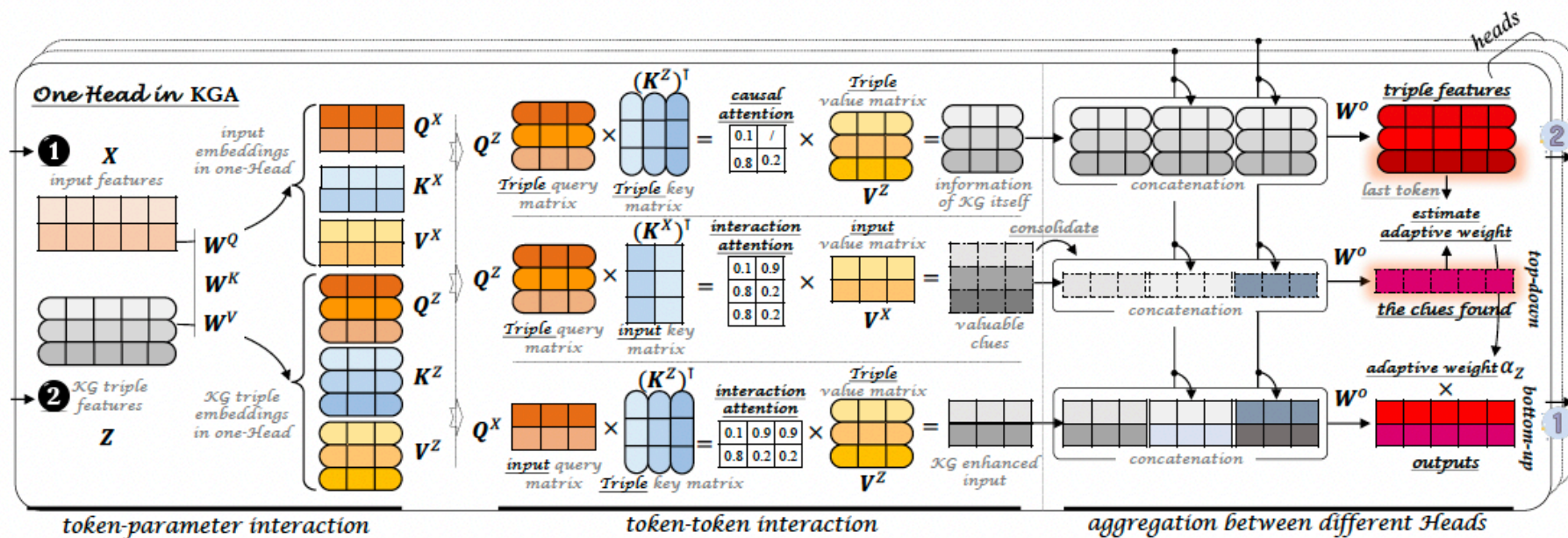
- The base model $\mathbf{M}$ receives a text prompt $\mathbf{X}$ (token sequence).
- External knowledge is a set of candidate triples $\mathbf{G}$
 - each triple $\mathbf{Z} = (\text{subject}, \text{relation}, \text{object})$.
- The integration mechanism must:
 - incorporate triples into the forward pass
 - do so **dynamically** (query-specific)
 - do so **without introducing new trainable modules**
- The output is the usual LLM output $\mathbf{Y}$ for the prompt.

THE “REWIRING”: REPLACE VANILLA SELF-ATTENTION WITH KGA

- Key engineering approach:
 - **Keep:** the transformer architecture,
 - But **replace:** the vanilla attention, inside each layer, with “KGA attention”.
- **Rewiring attention is sufficient:** no need to “train new components”
- Essentially, the mental model is:
 - Same projection matrices (W^Q , W^K , W^V),
 - But attention now routes between **two spaces**:
 - token space (input sequence)
 - triple space (KG triples as sequences)

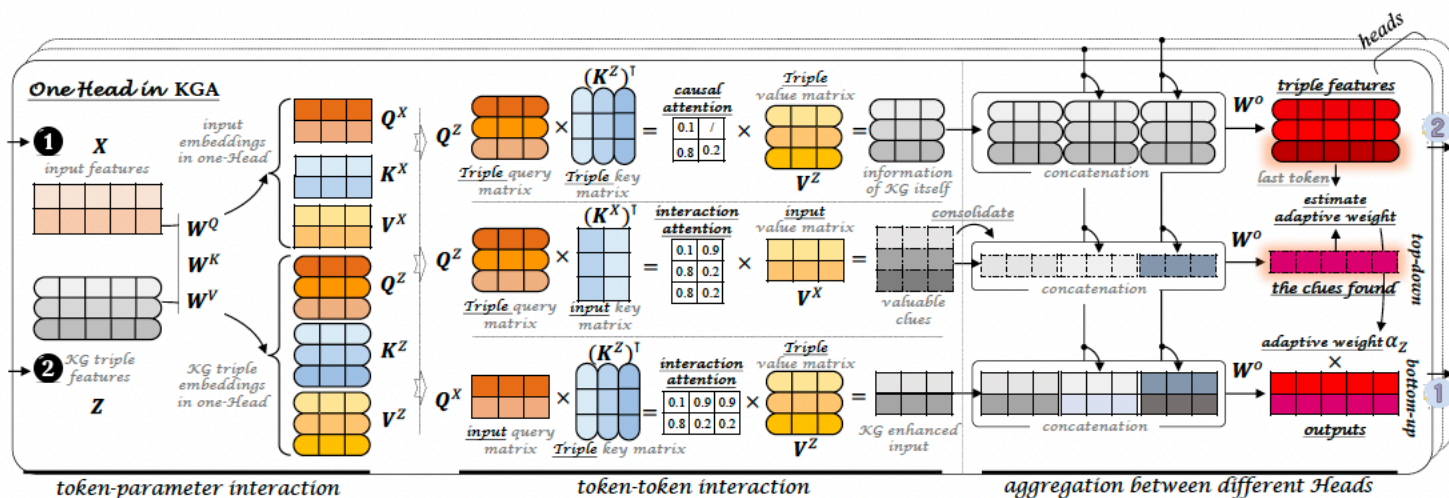
IN THE TRANSFORMER





THREE INTERACTION TYPES INSIDE A LAYER

- KGA integrates three essential fusion dimensions:
 - **Input-Itself** (token $\leftrightarrow$ token): normal self-attention keeps linguistic coherence.
 - **Triple-Itself** (within a triple): triple token self-attention consolidates meaning of the triple.
 - **Triple-to-Input / Input-to-Triple**: controlled information exchange between KG and text.
- Versus ICL and traditional cross-attention.



HOW TRIPLES ARE REPRESENTED: TRIPLES ARE TOKEN SEQUENCES TOO

- Each triple Z is treated as a **small token sequence**
 - (subject tokens + relation tokens + object tokens).
- That means:
 - a triple has its own internal token-level structure
 - we can run attention *within* a triple !
- Why that matters:
 - relations and entity names are multi-token (e.g., “held_in”, “New York City”)
 - triple self-attention helps create a coherent triple-level representation.

BOTTOM-UP PATHWAY: “TOKENS PULL KNOWLEDGE”

- Intuition: each input token asks - *“which triple tokens contain information useful to me right now?”*
- Mechanism in attention terms:
 - token queries attend over triple keys
 - token collects a weighted mixture of triple values
- Output:
 - each token representation is enriched with knowledge content relevant to it.
- Stimulus-driven: the *query* (input representation) determines what knowledge is extracted.

$$\hat{x}_n = \underbrace{\sum_{i=1}^N \frac{\text{Exp}(w_{n,i}^X) \cdot v_i^X}{\sum_{j=1}^N \text{Exp}(w_{n,j}^X)}}_{\text{aggregation: input itself}} + \underbrace{\sum_{Z \in \mathcal{G}} \alpha_Z \sum_{m=1}^M \frac{\text{Exp}(r_{n,m}^Z) \cdot v_m^Z}{\sum_{j=1}^M \text{Exp}(r_{n,j}^Z) / \sqrt{D}}}_{\text{aggregation: external knowledge}}$$

BOTTOM UP PATHWAY

- Use the **same Q/K/V projection matrices** as the base transformer:
 - triple tokens are projected by the model's original $W^Q/W^K/W^V$
 - so triple vectors land in the same semantic space as token vectors
- The we get meaningful semantic interaction **without adding new learned parameters**
- This is the “non-invasive” part: no new projection heads; reuse existing.

$$Q^Z = ZW^Q, \quad K^Z = ZW^K, \quad V^Z = ZW^V$$

BOTTOM-UP ALONE IS PROBLEMATIC

- In practice, candidate triples $\mathbf{G}$ can be noisy:
 - retrieval via 1-hop / 2-hop neighbors can bring irrelevant triples
- Bottom-up alone risk:
 - token similarity might pull distractor triples (“semantic look-alikes”)
 - the model starts reasoning from the wrong injected info
- So KGA needs a **gating signal**:
 - “how important is triple Z for this question?”
- This is where top-down enters !

TOP-DOWN PATHWAY: “TRIPLES VERIFY RELEVANCE”

- Top-down computes *adaptive weight* α_z via three stages:
 - **1. Clue extraction:** a triple attends to the input to find evidence (“clues”).
 - **2. Clue consolidation:** combine token-level clues into one triple-level clue vector.
 - **3. Relevance scoring:** score triple relevance and normalize into α weights.
- Each triple asks: “do I see anything *in the question* that makes me useful?”

STAGE 1: CLUE EXTRACTION (TRIPLE → INPUT ATTENTION)

- For each token in a triple:
 - use triple-token query to attend over all input tokens
 - produce a “clue vector” summarizing input evidence viewed through that triple token
- Triple tokens act like detectors:
 - subject token looks for subject mentions / aliases
 - relation token looks for relation cues (“located in”, “born in”)
 - object token looks for candidate target constraints
- Result: a set of clue vectors per triple-token, each highlighting different aspects.

$$c_m = \underbrace{\sum_{i=1}^N \left\{ \text{Exp}\left(\frac{q_m^Z(k_i^X)^\top}{\sqrt{D}}\right) v_i^X \right\}}_{\text{extracting a clue from } X \text{ guided by triple token } z_m} / \left\{ \sum_{j=1}^N \text{Exp}\left(\frac{q_m^Z(k_j^X)^\top}{\sqrt{D}}\right) \right\}$$

STAGE 2: CLUE CONSOLIDATION (TOKEN-LEVEL → TRIPLE-LEVEL)

- Problem:
 - not all triple tokens are equally informative
 - naive averaging can dilute the key token's signal
- Solution:
 - use the triple's *own internal attention* to weight clue tokens
 - specifically, use attention from the **last triple token** to others as a learned importance signal
- The triple's structure itself *decides* which part matters most for meaning in context !

$$\hat{c} = w_M^Z C = \sum_{m=1}^M w_{M,m}^Z c_m$$

STAGE 3: RELEVANCE SCORING AND ADAPTIVE WEIGHT

- After consolidation, compute a scalar relevance score $s(Z, X)$.
- Convert these into normalized weights α_Z over candidate triples:
 - softmax normalization across all Z in G
- Controlled by temperature τ :
 - higher $\tau \rightarrow$ softer distribution (use more triples)
 - lower $\tau \rightarrow$ sharper (focus hard on top triples)
- α_Z is “how much this triple deserves to influence the answer”.

$$s(Z, X) = \hat{c} (z_M)^\top \quad \alpha_Z = \frac{\text{Exp}(s(Z, X)/\tau)}{\sum_{Z' \in G} \text{Exp}(s(Z', X)/\tau)}$$

HOW TOP-DOWN AND BOTTOM-UP COMBINE IN THE FINAL TOKEN UPDATE

- Bottom-up provides the mechanism: token pulls triple content.
- Top-down provides the modulator: α_Z scales each triple's contribution.
- So token update becomes:
 - “my usual token-to-token attention output”
 - **plus** “a weighted blend of triple information”
 - where the weights are learned *per query* via top-down verification.
- Practical interpretation:
 - relevant triples get amplified,
 - noisy triples get suppressed.

$$\hat{\mathbf{x}}_n = \underbrace{\sum_{i=1}^N \frac{\text{Exp}(w_{n,i}^X) \cdot \mathbf{v}_i^X}{\sum_{j=1}^N \text{Exp}(w_{n,j}^X)}}_{\text{aggregation: input itself}} + \underbrace{\sum_{Z \in \mathcal{G}} \alpha_Z \sum_{m=1}^M \frac{\text{Exp}(r_{n,m}^Z) \cdot \mathbf{v}_m^Z}{\sum_{j=1}^M \text{Exp}(r_{n,j}^Z) / \sqrt{D}}}_{\text{aggregation: external knowledge}}$$

WHAT MAKES KGA DIFFERENT FROM CROSS-ATTENTION AND ICL

- KGA's "tripartite" point:
 - Retains Input-Itself attention (keeps language coherence)
 - Retains Triple-Itself consolidation (keeps triple semantics intact)
 - Adds controlled Triple $\leftrightarrow$ Input flow (knowledge exchange)
- In-context learning (ICL):
 - creates inter-triple interference and ordering sensitivity
 - wastes computation on triple-to-triple interactions that are meaningless noise
- Traditional cross-attention: tends to be unidirectional

ALGORITHM

- Embed tokens for X and tokens for each triple Z .
- For each layer:
 - estimate triple relevance α_Z using top-down:
 - clue extraction $\rightarrow$ consolidation $\rightarrow$ scoring $\rightarrow$ softmax
 - perform bottom-up fusion:
 - update X using token self-attention + triple attention scaled by α_Z
 - update triple representations as they pass through layers (so relevance can evolve)
- Output generation proceeds normally from the enriched token stream.

Algorithm 1 Process of language models equipped with KGA

Require: Candidate triples $\mathcal{G}$, Query X , temperature τ .

Ensure: Model predictions $\mathcal{M}_{\Theta}(X)$ based on our KGA.

```
1: // Initialize representations of  $Z \in \mathcal{G}$ ,  $X$  from token embeddings
2:  $Z^{(0)}, X^{(0)} \leftarrow \text{TokenizeAndEmbed}(Z, X)$ 
3: // Model Forward Pass
4: for  $layer\_idx$  in  $\text{enumerate}(\text{hidden\_layers})$  do
5:   // 1. Estimate triple relevance
6:   // 1.1. Extract valuable clues
7:   Estimate  $C$  via Eq. 3 based on  $X$  and  $Z$ 
8:   // 1.2 Perform forward pass of triples
9:    $Z^{(l+1)} \leftarrow \text{KGA\_layer}(Z^{(l)})$ 
10:  // 1.3 Consolidate clues and compute relevance
11:  Consolidate and perform  $\alpha_Z$  estimation via Eq. 4 - Eq. 6
12:  // 2 Adaptive integration
13:  Perform knowledge fusion via Eq.2
14:   $X^{(l+1)} \leftarrow \text{KGA\_layer}(X^{(l)})$ 
15: end for
16: Logits  $\leftarrow \text{LMHead}(X^{(L)})$ 
17: return Decode(Logits)
```

EVALUATION: SETUP

- Evaluate KGA on **three distinct tasks**:
 - **KGQA** (knowledge graph question answering)
 - given a question + candidate triples → answer
 - **KG Reasoning**
 - questions requiring multi-hop reasoning over triples (e.g., 2-hop paths)
 - **Knowledge-based Model Editing (KME)**
 - the model must incorporate new facts continuously (consecutive edits)
 - simulates a real dynamic knowledge environment
- Key detail about candidate triples G:
 - They avoid full-KG input (too expensive).
 - They avoid heavy retrieval systems (to not confound results).
 - They do entity linking on question mentions, then retrieve 1-hop or 2-hop neighboring triples.
- This intentionally introduces noise, testing robustness.

KGQA + KG REASONING

- Benchmarks used:
 - SimpleQuestions** (KGQA)
 - Think: “single-fact” questions
 - evaluation question form:
 - given a natural language question, pick the correct answer entity supported by one key triple
 - metric: Hit@1
 - PathQuestion (2-hop)** (KG reasoning)
 - questions where answer requires following a 2-hop path in the KG
 - evaluation question form:
 - use two linked triples (subject→mid, mid→object) to reach answer
 - metric: Hit@1
- Difference
 - KGQA: “find the right triple”
 - reasoning: “find the right chain of triples”

<i>(KGQA)</i>		<i>(KG Reasoning)</i>	
SimpleQuestions		PathQuestion (2-hop)	
<i>Methods</i>	<i>Hit@1</i>	<i>Methods</i>	<i>Hit@1</i>
MemNN [2]	63.9	ISM [28]	99.1
CFO [9]	62.6	QAGCN [58]	98.5
AMPCNN [67]	67.2	Uhop-HR [5]	97.6
Character [17]	70.3	KV-MemNN [41]	97.4
IOPrompt [3]	20.0	SRN [44]	96.3
SC [61]	18.9	IRN [74]	96.0
RoG [36]	73.3	MINERVA [10]	75.9
StructGPT [†] [21]	50.2	TransferNet [48]	93.2
PoG [*] [4]	63.9	RL-MHR [11]	94.1
KnowPath [*] [71]	65.3	AlAgha [5]	97.4
ToG [*] [49]	59.7	ARN_DistMult	84.3
ZSL (Qwen2.5 0.5B)	19.5	ZSL (Qwen2.5 0.5B)	20.1
ICL (Qwen2.5 0.5B)	67.3	ICL (Qwen2.5 0.5B)	73.7
KGA (Qwen2.5 0.5B)	75.1	KGA (Qwen2.5 0.5B)	98.3

BASELINES

- Three baseline groups
 - **SFT (Supervised Fine-Tuned)**: train on KG-augmented data
 - **ICL (In-Context Learning)**: flatten triples into text and concatenate with question
 - **ZSL (Zero-Shot Learning)**: no KG integration, just the LLM's internal knowledge
- **Advanced inference-time methods**: RoG, PoG, ToG
- For KME (model editing), compare to:
 - finetuning approaches (FT-C, LoRA)
 - parameter editing methods (ROME, AlphaEdit, etc.)
 - memory-based methods (GRACE, WISE)
 - ICL-style editing baseline (IKE)
- Different baselines represent different philosophies:
 - “change weights”
 - “stuff context”
 - “edit parameters surgically”
 - “attach external memory”

RESULTS

- **KGQA + KG Reasoning (Table 1)**
 - KGA beats ZSL by a huge margin (expected: ZSL doesn't see KG).
 - The meaningful comparison: KGA vs ICL and vs advanced inference-time methods.
- On **PathQuestion (2-hop)**, KGA is near the top end (paper reports ~98+ Hit@1 for their Qwen2.5-0.5B setting).
 - Interpretation: multi-hop questions benefit disproportionately because structured triple integration matters more than prompt flattening.
- **Generalization across LLMs (Table 2)**
 - KGA works across Llama3.2 (1B/3B), Llama3 (8B), Qwen2.5 (1.5B/3B/7B).
 - suggests the method is not model-idiosyncratic;
 - it leverages “universal” attention mechanics.
- **Model editing (Table 3, 3K consecutive edits)**
 - KGA performs strongly on:
 - Efficacy (did the edit take?)
 - Generality (does the edit generalize across paraphrases/contexts?)
 - Locality (did we avoid breaking unrelated facts?)
- Basically, since KGA injects knowledge dynamically rather than overwriting weights, it can do well on “locality” in a dynamic update stream.